Ruggine

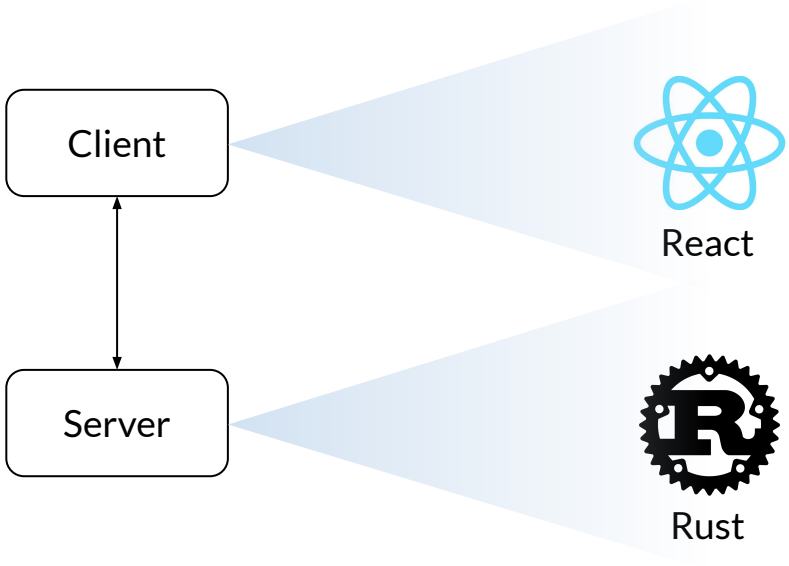
Alessio Perrotti: 356737
Alberto Nuzzaci: 346977
Riccardo Vaccari: 348856
Luca Lodesani: 346978

Obiettivi del progetto

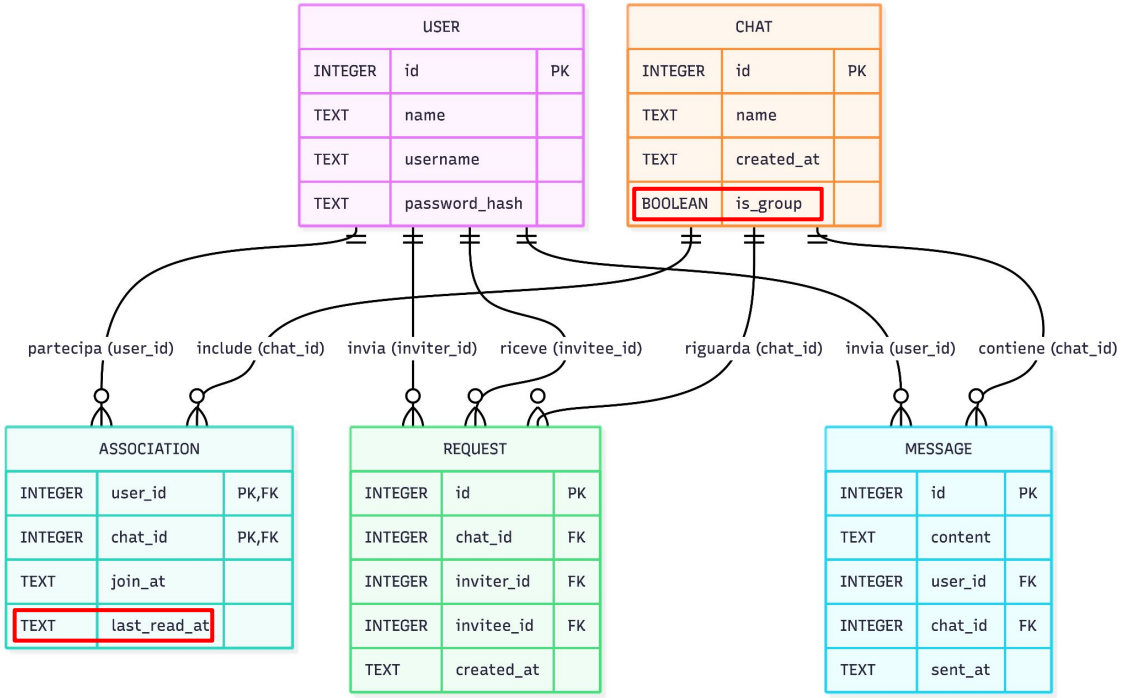
Ruggine è un app di chat in tempo reale con le seguenti funzionalità:

- Chat Private & Chat di Gruppo
- Inviti alla partecipazione di un gruppo
- Messaggi in tempo reale tra i membri della chat
- Accettazione/Rifiuto invito a un gruppo
- Abbandono di un gruppo

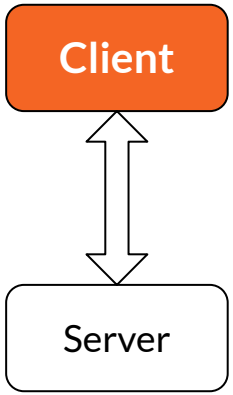
Architettura



Architettura - Database



Architettura - Client Side



Reattività State-Driven

Sincronizzazione automatica UI-Dati per aggiornamenti real-time fluidi

Stato Globale

Centralizzazione dei dati per garantire coerenza

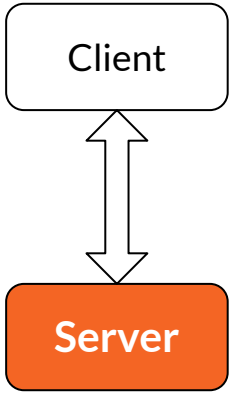
Modularità via Custom Hooks

Logica applicativa isolata in moduli riutilizzabili e snelli

Cross-platform & Portabilità

Accesso universale via browser e codice pronto per Mobile/Desktop

Architettura - Server Side



Memory Safety

Sicurezza della memoria garantita senza Garbage Collector

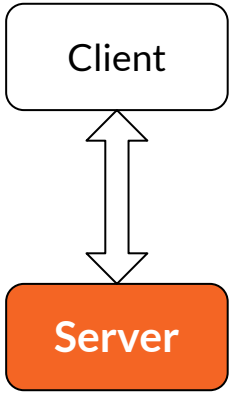
Fearless Concurrency

Gestione sicura delle connessioni simultanee

Performance

Minimo consumo di risorse

Architettura - Server Side



Tokio

Runtime per operazioni asincrone e I/O non bloccante

Axum

Framework web per gestire API REST e WebSocket

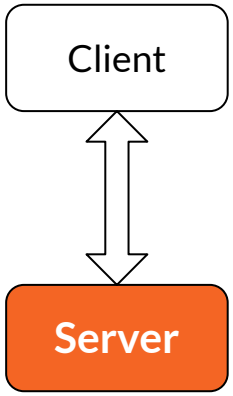
SQLite

Database relazionale leggero

SysInfo

Monitoraggio delle presentazioni della CPU

Architettura - Stato Globale nel Server



Routing con Axum

`AppState` viene estratto e clonato per ogni richiesta HTTP concorrente

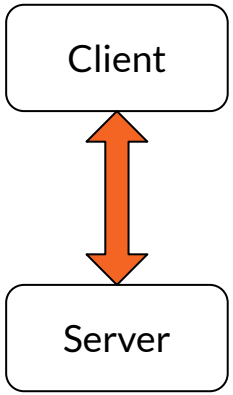
Gestione della memoria

Campi pesanti o mutabili non vengono copiati, ma incapsulati in **Smart Pointers**

Thread Safety

Utilizzo di Arc per garantire che i riferimenti sopravvivano nei task asincroni gestiti dal runtime di Tokio

```
pub struct AppState {
  pub pool: SqlitePool,
  pub cookie_secret: Vec<u8>,
  pub session_ttl_secs: u64,
  pub notification_peers: Arc<RwLock<HashMap<...>>>,
  pub next_conn_id: Arc<AtomicU64>,
}
```



Architettura - Comunicazione ibrida

API REST (Azione & Storico)

Ruolo → Operazioni **sincrone** (Client → Server)
Cosa fa → Login, invio dei messaggi, fetch storico chat e messaggi
Perché → Sfrutta il **routing Axum**, gestione nativa errori HTTP e validazione sicura tramite Extractors

WebSocket (solo notifiche)

Ruolo → Eventi **real-time** (Server → Client)
Cosa fa → Push di eventi asincroni (es. ricezione messaggi, inviti utenti, ...)
Perché → **Unidirezionale** (read-only per il client), evita complessità nel server

Server - Il flusso di comunicazione

1. Handshake & Split

La connessione HTTP fa l'upgrade. Il WebSocket viene diviso in `ws_sender` e `ws_receiver`

```rust
pub async fn ws_notifications(
    ws: WebSocketUpgrade,
    State(state: AppState): State<AppState>,
    auth: AuthUser,
) -> impl IntoResponse {
    let conn_id: u64 = state.next_conn_id.fetch_add(val: 1, order: Ordering::Relaxed);
    ws.on_upgrade(callback: move |socket: WebSocket| handle_notifications_socket(socket, state, auth.user_id, conn_id))
}
```

Server - Il flusso di comunicazione

2. Registrazione
Creazione del canale MPSC. Il `Tx` viene salvato in `notification_peers`

Anatomia di
`notification_peers`

```
Arc<RwLock<HashMap<i64, HashMap<u64, UnboundedSender<Message>>>>>
```

Ottimizzazione per la **concorrenza asincrona**:
Read → invio dei messaggi (parallelo)
Write → connessione/disconnessione

`id` di un utente con almeno una connessione attiva

`id` della connessione generato con `AtomicU64`

`Sender` del canale MPSC attraverso cui inviare il messaggio WebSocket

Enum di Axum che rappresenta un messaggio WebSocket

Server - Il flusso di comunicazione

3. Broadcast

Un utente invia un messaggio tramite richiesta HTTP POST; il server recupera i `Tx` dalla mappa e invia il payload

```rust
/// Broadcasts a WebSocket message to every user in the provided slice.
pub async fn notify_users(&self, user_ids: &[i64], msg: Message) {
    let peers: RwLockReadGuard<'_, HashMap<…, …>> = self.notification_peers.read().await;
    for &uid: i64 in user_ids {
        if let Some(senders: &HashMap<u64, UnboundedSender<…>>) = peers.get(&uid) {
            for tx: &UnboundedSender<Message> in senders.values() {
                let _ = tx.unbounded_send(msg.clone());
            }
        }
    }
}
```

Server - Il flusso di comunicazione

4. Task di forwarding

Un thread dedicato, in attesa, legge dal `Rx` del canale e scrive fisicamente sul `ws_sender`

```
// Forward messages from the internal channel to the WebSocket.
let forward: JoinHandle<()> = tokio::spawn(future: async move {
    while let Some(msg: Message) = rx.next().await {
        if ws_sender.send(item: msg).await.is_err() {
            break;
        }
    }
});
```

Server - Il flusso di comunicazione

5. Cleanup

Se il socket si chiude, il task muore e l'ID viene rimosso in modo sicuro dalla mappa

```
let mut peers: RwLockWriteGuard<'_, HashMap<…, …>> = state.notification_peers.write().await;
if let Some(conns: &mut HashMap<u64, UnboundedSender<…>>) = peers.get_mut(&user_id) {
    conns.remove(&conn_id);
    if conns.is_empty() {
        peers.remove(&user_id);
    }
}
```

Server - L'utilizzo dei canali MPSC

In memoria

L'invio sul canale è in RAM, istantaneo e non bloccante

Ownership semplificata

Il server richiede solo un reference immutabile ed e facile da clonare

Disaccoppiamento I/O

Un thread isolato consuma i messaggi nel canale ed esegue l'invio asincrono sul ws

Prestazioni e Dimensioni

Misurazione asincrona, eseguito da un thread separato, ogni 2 minuti.
Salvataggio dei dati su un file di log.

```
[2026-04-18 08:08:26.414469438 UTC] CPU usage: 1.28%
[2026-04-18 08:10:26.417537014 UTC] CPU usage: 0.10%
[2026-04-18 08:12:26.421488479 UTC] CPU usage: 0.02%
[2026-04-18 08:14:26.424727661 UTC] CPU usage: 0.01%
[2026-04-18 08:39:54.079698708 UTC] CPU usage: 0.00%
[2026-04-18 08:41:54.082709129 UTC] CPU usage: 1.00%
```

Dimensioni dell’eseguibile (lato server):

MacOS	6.8 MB
Ubuntu	8.1 MB

Grazie dell’attenzione

Alessio Perrotti: 356737
Alberto Nuzzaci: 346977
Riccardo Vaccari: 348856
Luca Lodesani: 346978